COLLAGE OF COMPUTER SCIENCE & ENGINEERING

UNIVERSITY OF JEDDAH

كلية علوم و هندسة الحاسب

جامعة جدة

JS4Backend: Server–side JavaScript

CCSW 321 (Web Development)

What will be covered

- Node.js Introduction
- Node.js Routing
- Database Operations.
- Backend Data Validation with Node.js

Node.js Intro

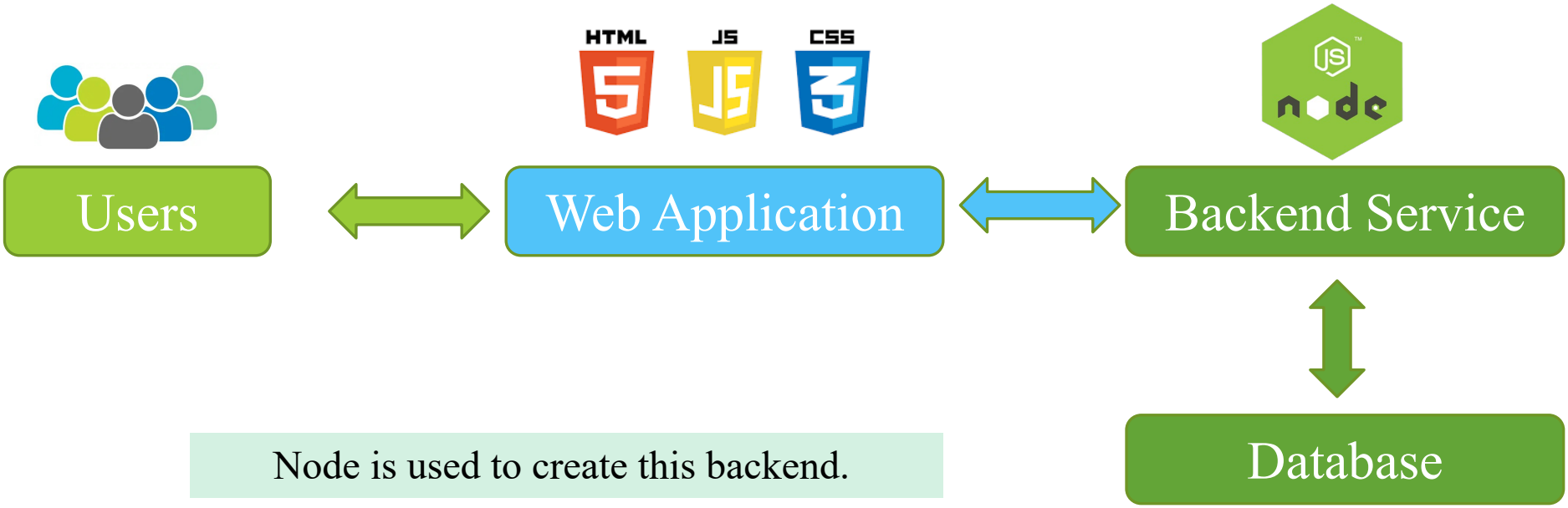
What is Node.js?

- Node.js is an open-source, server-side JavaScript runtime environment built on Chrome's V8 JavaScript engine.
- It allows developers to run JavaScript code **outside of a web browser.**
- Node runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node is mainly used to create **back-end services**.

Node.js Intro

What are backend services?

- To process business logic and conduct data operations, we need a backend that offer such services to our app.



Node.js Intro

Node.js provides a non-blocking, event-driven architecture.

- It uses an **event loop** to handle multiple concurrent requests without blocking the execution of other code.
- This **asynchronous nature** of Node.js allows it to **handle a large number of concurrent connections** efficiently, making it well-suited for applications that require real-time communication or deal with heavy I/O operations.

Node.js Intro

What can we do with Node.js?

Handle business and data logic, e.g.,:

- Generate **dynamic page content**.
- Create, open, read, write, delete, and close **files on the server**.
- Collect **Form** data.
- Add, delete, modify data in your **database**.
- Create an **API** for our web application.

Node.js Intro

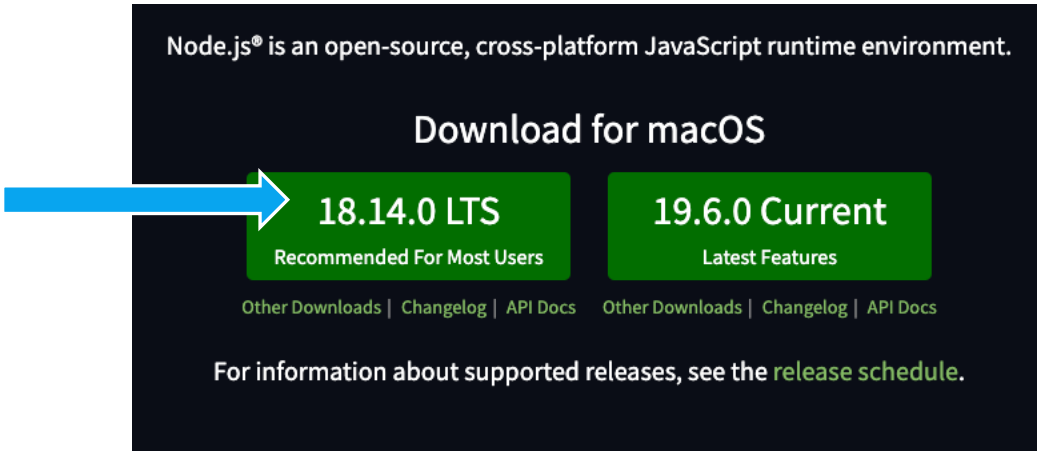
Who uses Node.js?

- Node.js is widely adopted by numerous companies, ranging from small startups to large enterprises. Some notable companies that use Node.js in their technology stack include:

Node.js Intro

Node Installation

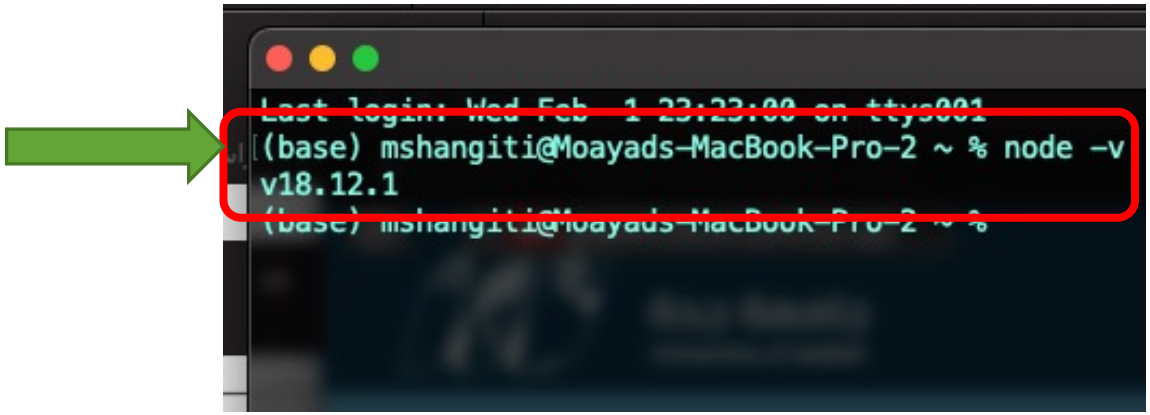
- To install node, visit https://nodejs.org
- Download and install the LTS version.



Node.js Intro

Node Installation

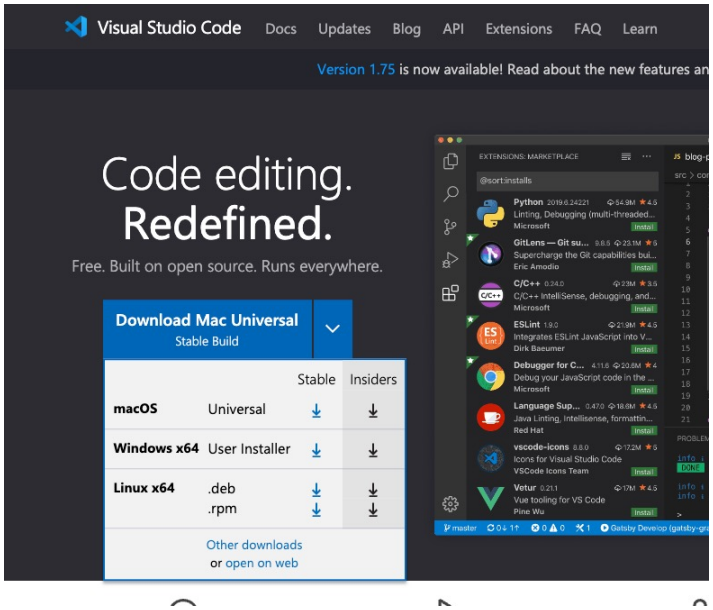
- Once installation is complete, open terminal (Mac) or cmd (windows) and test it by running the command "node -v".
- If the command returned the current installed Node version, then you have correctly installed Node.js



Node.js Intro

Node Installation

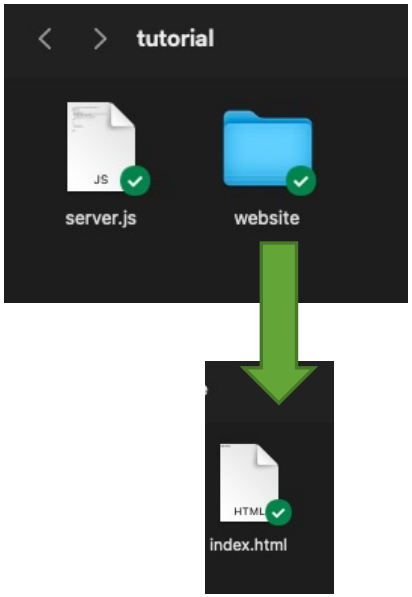
- Now that node is installed, let's try to run the first JavaScript code outside the browser! However, it is strongly recommended that you first install Visual Studio Code on your machine https://code.visualstudio.com/.



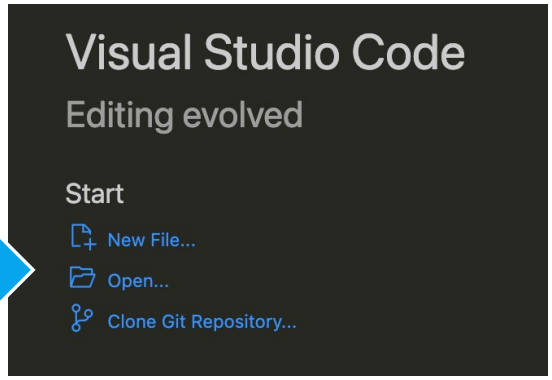
Node.js Intro

Hello World Example.

- Create a new folder called ‘tutorial’ and inside it of it create the following:
  - **server.js** = This will be our main javascript file that contains all our back-end code.
  - **website folder**= This will contain all our front-end code files. For our example, it will have:
  - **index.html** = HTML file with <h1>Hello World</h1>



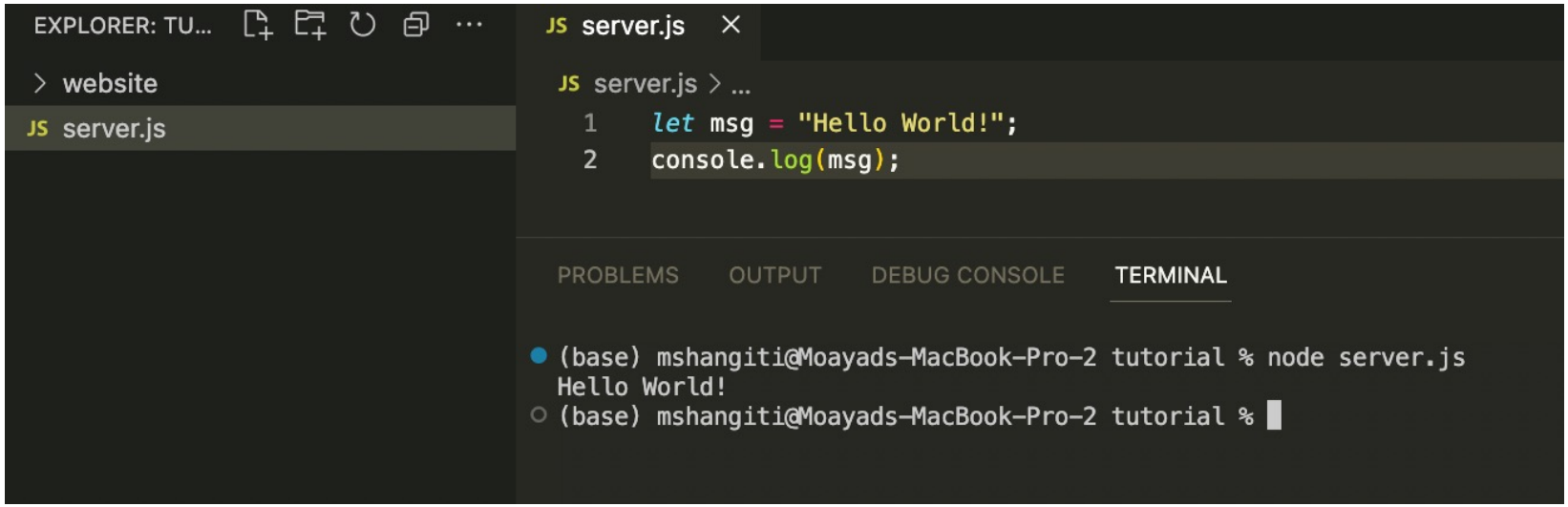
Open the folder inside visual studio code



Node.js Intro

Hello World Example.

- Let's code!
- Open the server.js file and write the following code in it:
- Now open the terminal and run the JavaScript file using node:



As you can see JavaScript is running **outside** the browser!

Node.js Intro

How can we run a web app using Node.js?

- First, let's setup the environment with **Node Package Manager (NPM)**.
  - **npm** is a package manager that comes with Node.
  - **npm** helps us install **JavaScript** **modules** with simple commands.
  - **Modules** are the same as libraries, they are simply a set of functions you want to include in your application.
- With the help of npm, we will install three libraries in our project folder:
  - **Express** = Manages routing and communication between front-end and back-end.
  - **Express-validator** = Makes user data validation much easier.
  - **Mysql2** = Manages connection to the database.

Node.js Intro

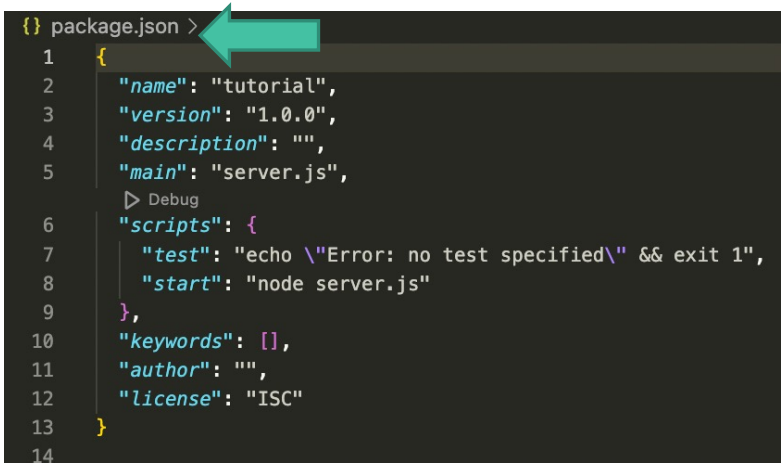
How can we run a web app using Node.js?

- We add the package manager to our project's folder.
- Run the following command in the terminal:

```
npm init --yes
```

The flag '—yes' will initialize the project with the default values.

This command will create a 'package.json' file that contains information on our project.



Node.js Intro

How can we run a web app using Node.js?

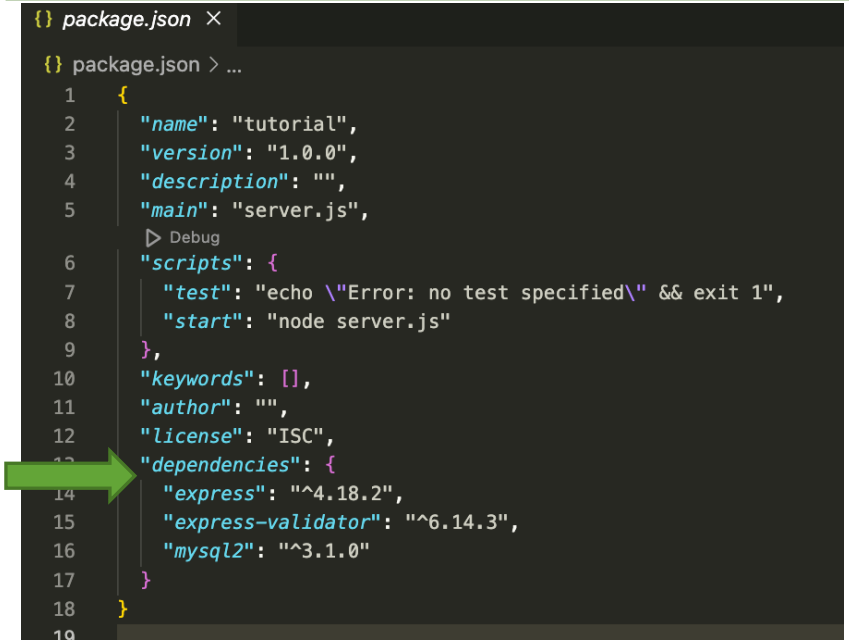
- Now that we added the package manager (npm), we can use it to install our libraries by running the following command **in the terminal**:

```
npm install express
```

```
npm install express-validator
```

```
npm install mysql2
```

Once we have installed the libraries, we can see them added as dependencies in the package.json file



Node.js Intro

How can we run a web app using Node.js?

- Second, let write the necessary Node.js code to start a backend server.

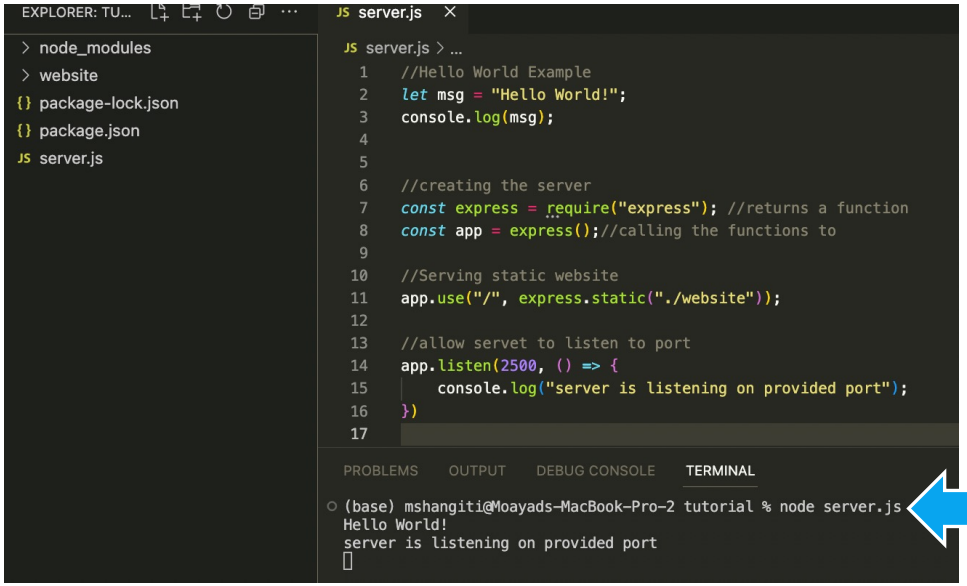
- First, we establish a reference with the ***express*** module that helps us manage communication between front-end and back-end.
- Second, we let ***express*** know the location of our website folder so that it can serve it when server is running.
- Finally, we activate the server and have it listen to events on a specific port, e.g., in here it is port 2500.

Node.js Intro

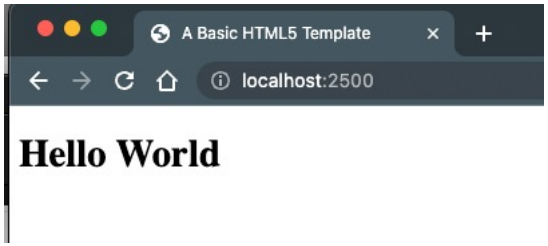
How can we run a web app using Node.js?

- Now we run the JS code by executing the following command in the terminal:

```
node server.js
```



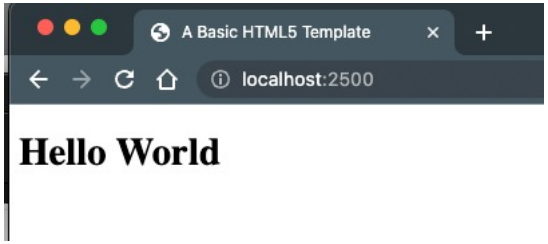
Open browser to http://localhost:2500



Node.js Intro

How can we run a web app using Node.js?

- We now have a running server that will server all the files inside the 'website' folder.
- Try http://localhost:2500/index.html
- Try adding a new html file for the website folder and navigate to it.
- If you **make any changes to server.js**, you will need **to stop the server** from the terminal by clicking on "control" and the letter "c", then **run node again**. Otherwise, you will not see your changes.
- Note that the ***port number*** in the **address MUST match** the port number specified in the **code**.

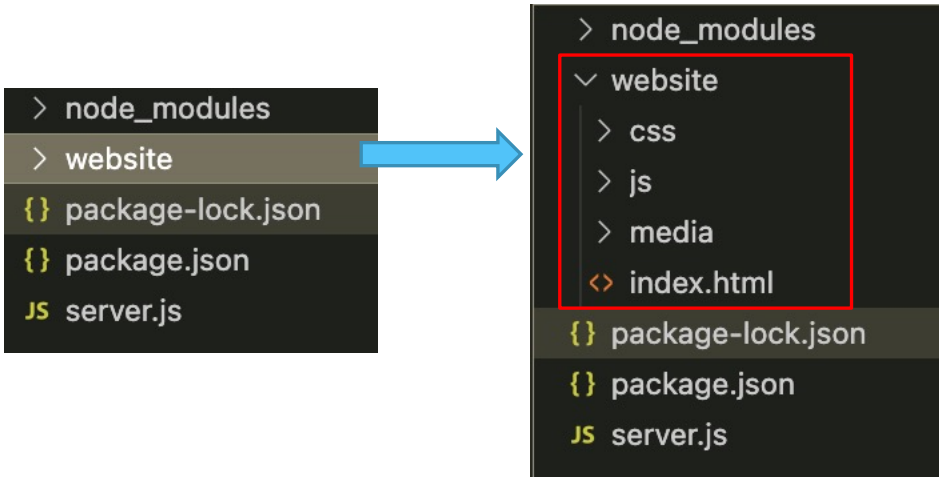


Node.js Intro

How can we run a web app using Node.js?

Summary of project's folder:

- **server.js** = will hold all our **backend code and logic**.
- **Website folder**= will hold all our **frontend code (HTML,CSS, etc)**.
- What about **node_modules, package-lock.json, package.json**? Ignore those files. They are used by the package manager NPM to manage dependencies.



Node.js Routing

Going beyond basic website serving.

- Now that we have covered the basic setup of serving a website using Node.js, let's dive into the valuable features it offers. We'll explore creating a simple web service to handle:
  - **business logic**.
  - performing **database operations.**
  - implement **backend data validation** to enhance the security of our website.
  - Let’s start with explaining the **meaning of routing.**

Node.js Routing

- **Node.js routing** refers to the process of defining and managing different routes in a Node.js application. Routing allows us to handle different HTTP requests and **direct them to the appropriate code handlers** based on the requested URL and HTTP method.
- Create a structured and organized backend application by separating different functionalities into modular routes. **For example**, we can have separate routes for handling **user authentication**, **data retrieval**, **data creation**, or any other specific functionality. This approach helps in keeping the codebase maintainable and allows for easier scalability as the application grows.

Node.js Routing

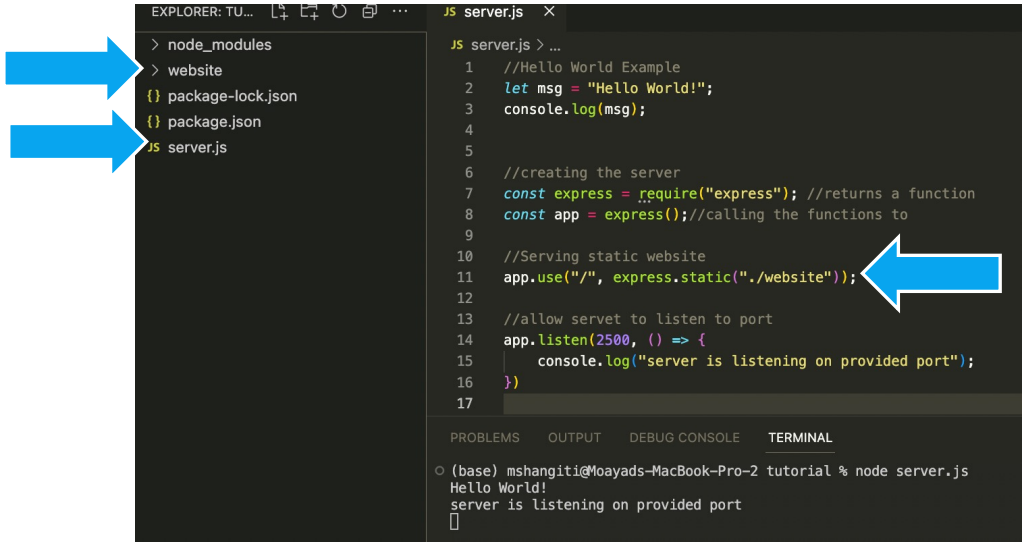
We can **divide** routing **by purpose** to the following:

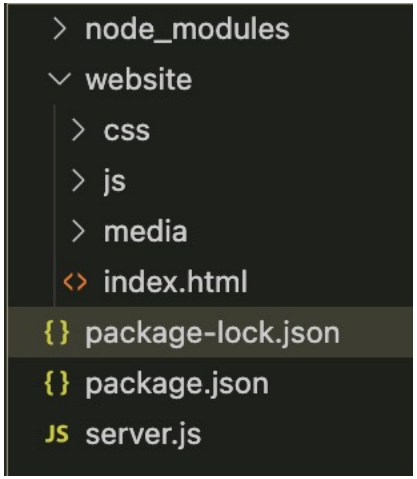
- **Static Routing**: used to serve **static** files such as HTML, CSS, JavaScript, images, and other assets. It involves mapping specific URLs to corresponding file paths in the file system.
- **HTML Routing**: used to **dynamically** generate HTML content and send it back to the client. It involves defining routes that generate HTML templates, render dynamic data into the templates, and send the resulting HTML to the client.
- **JSON Routing**: used for building **API-like services** where data is exchanged in JSON format.

Node.js Routing

Static Routing

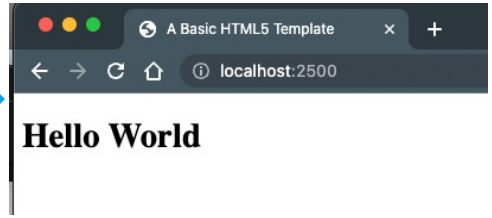
- We have already seen this type of routing.
- We have statically served the **website folder** that included index.html, style.css, script.js





The **server.js** program checks the **website directory** to see if "index.html" exists, which is does, so it replies with that file.

Now when you start the server and visit **localhost:2500**, you will be **routed** to index.html

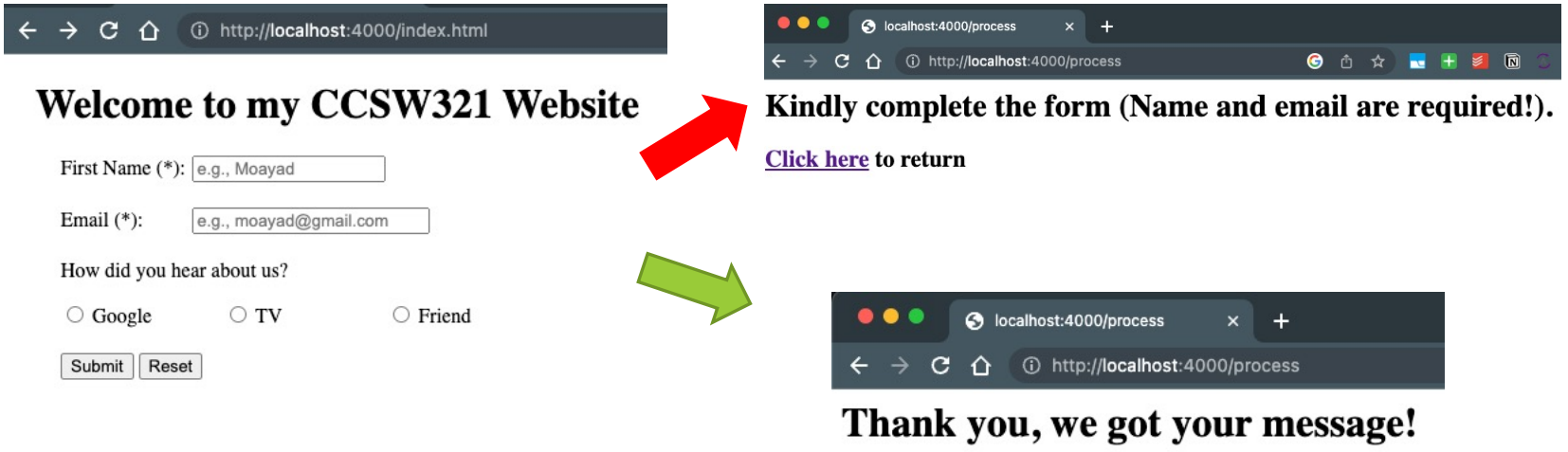


Make sure to name your landing page index.html

Node.js Routing

HTML Routing

- Static routing is great for serving a static website, but how about if we want to **dynamically change the page content**?
- E.g., How can we handle a form submission that would process the request and then display a thank you page or an error page?



Node.js Routing

Index.html

- Create a form.html page with the form shown on the right side.
- Make sure all <input> fields are given a ‘name’ attribute.
- Make sure the <form> **method attribute** is set.
- Make sure the <form> **action attribute** is set.
- The **method** and **action** must match Node.js routing.

```
<body>
<div id='wrapper'>

  <h1>Welcome to my CCSW321 Website</h1>
  <form method="POST" action="/process">
    <section>
      <!-- First name -->
        <div>
          <label for='fname'>First Name (*):</label>
          <input type="text" name="fname" placeholder="e.g., Moayad" >
        </div>
      <!-- email -->
        <div>
          <label>Email (*):</label>
          <input id='email' type="email" name="email" placeholder="e.g.,
moayad@gmail.com" size='25' minlength="1" maxlength="100">
        </div>
        <!-- How did you hear about? -->
        <div id='hear'>
          <p>How did you hear about us?</p>
              <input type="radio" name="aboutus" value='Google'>
              <label>Google</label>

            <input type="radio" name="aboutus" value='TV'>
           <label>TV</label>

              <input type="radio" name="aboutus" value='Friend'>
              <label>Friend</label>
        </div>
    </section>
        <div>
          <p id='msg'></p>
          <input type="submit">
          <input type="reset">
        </div>
  </form>

</div>
</body>
```

Node.js Routing

server.js

```
//create the server
const express = require("express");
const app = express();

//static routing
app.use("/", express.static("./website"));

//HTML routing
app.use(express.urlencoded({ extended: false }));
//Handling POST request to /process
app.post("/process", (request, response) => {
  //reading form data
  const fname = request.body.fname;
  const email = request.body.email;
  let msg = "";
  //check if all name and are submitted
  if (fname.length > 0 && email.length) {
    msg = `<h1>Thank you, we got your message ${fname}! </h1>`;
  } else {
    msg =
      "<h1>Kindly complete the form (Name and email are required!).</h1><h2><a
href='index.html'>Click here</a> to return</h2>";
  }
  //send response
  response.send(msg);
});

//server
app.listen(4000, () => {
  console.log("Server is running.");
});
```

Static routing

HTML routing

- **Express.urlencoded()** is set to **false** to let the server know we are parsing data sent in the body of a POST request.
- App.**post**("/process",…) sets instructions on how to process **POST** requests sent to the link "/process".
- **Request** parameter contains parameters sent by the <form> submission.
- **Response** parameter allows us to reply back to the user.

Node.js Routing

JSON Routing

- You may have noticed that we are sending HTML code back with HTML routing and it is being loaded in a new page. **How about if we want to use AJAX to handle communication and show results in the same page?**
- To do so, we will need to send back a message in JSON format and process the message at the frontend.

Node.js Routing

server.js

```
//create the server
const express = require("express");
const app = express();

//static routing
app.use("/", express.static("./website"));

//JSON routing
app.use(express.json());
//Handling POST request to /process
app.post("/process", (request, response) => {
  //reading form data
  const fname = request.body.fname;
  const email = request.body.email;
  let msg = {};
  //check if all name and are submitted
  if (fname.length > 0 && email.length) {
    msg = { status: true, err: "" };
  } else {
    msg = {
      status: false,
      err: "Kindly complete the form (Name and email are
required!)",
    };
  }
  //send response
  msg = JSON.stringify(msg);
  response.json(msg);
});

//server
app.listen(4000, () => {
  console.log("Server is running.");
});
```

Set JSON mode on.

Construct an object message of key:value pairs to send back.

Encode response as JSON and send it back to client.

Node.js Routing

script.js

```
//event listener when submit button is clicked
document.getElementById("myform").addEventListener("submit",
function (event) {
  //prevent form from submission
  event.preventDefault();
  //reading user input
  let name = document.getElementsByName("fname")[0].value;
  let email = document.getElementsByName("email")[0].value;
  let aboutus = document.getElementsByName("aboutus")[0].value;

  //sending AJAX POST request to /process
  fetch("/process", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({ fname: name, email: email, aboutus:
aboutus }),
  })
    .then(function (response) {
      //check if any network issues exist
      if (response.ok) {
        return response.json(); // Parse the response as JSON
      } else {
        throw new Error("Ops,We had problems connecting to our
backend!");
      }
    })
    .then(function (result) {
      // if network issues, then Handle the parsed JSON data
      result = JSON.parse(result);
      //status is sent from backend
      if (result.status) {
        //status is true
        document.getElementById("msg").innerHTML =
          "<span style='color:green'>Thank you, your data has
been received.</span>";
      } else {
        //status is false, then show err msg
        document.getElementById("msg").innerHTML =
          "<span style='color:red'>" + result.err + "</span>";
      }
    })
    .catch(function (error) {
      //catch any thrown errors
      console.error("Error:", error);
      document.getElementById("msg").innerHTML =
        "<span style='color:red'>" + error + "</span>";
    });
});
```

Send POST request

Parse backend response

Node.js Routing

JSON Routing

- Complete AJAX interaction. No page refresh. No page redirection.
- Frontend and Backend communication is handled through AJAX and JSON messages.
- User feedback is provided in the same page.
- We can use **JSON routing** to create **our own web service API.**

Database Operations

Databases!

- **Databases** play a crucial role in web development, as they provide a structured way to store and retrieve data.
- Understanding databases is essential for building dynamic and data-driven web applications.
- In this section, we will explore the basics of databases, the most common operations, and how to connect Node.js with a MySQL database to handle all our data needs.

Database Operations

Databases!

- A database is a structured collection of data organized in a way that allows efficient storage, retrieval, and manipulation of data.

Common DB operations (Queries):

- **SELECT**: Retrieving data from the database based on specific criteria.
- **INSERT:** add new data in the database.
- **UPDATE**: Modifying existing data in the database.
- **DELETE**: Removing data from the database.

Database Operations

Databases!

- You should already be familiar with these SQL commands. This is a quick refresher on the four most common SQL queries:

```
SELECT * FROM users
```

```
INSERT INTO users (name, email)
    VALUES ('John Doe',
'johndoe@example.com');
```

```
    UPDATE users SET email =
'newemail@example.com' WHERE id = 1;
```

```
DELETE FROM users WHERE id = 1;
```

Database Operations

How can we store the user information to a MySQL Database?

- Create a table with the appropriate schema in MySQL.
- Create the addUser() function:
  - Open a database connection.
  - Create SQL query.
  - Execute query.
  - Close connection.
- Reply back to client.

Database Operations

Create a table with the appropriate schema in MySQL.

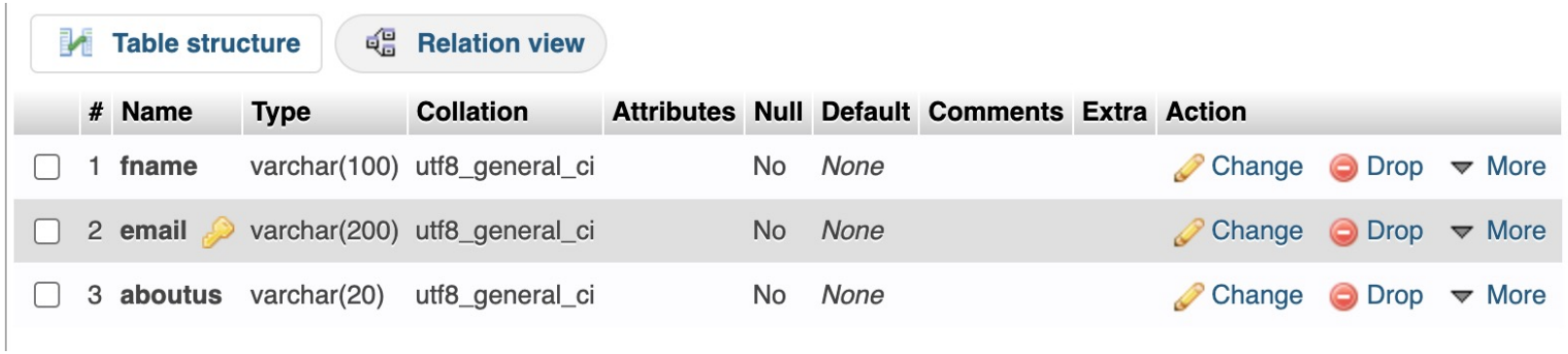
- To use a database, you need to install a database on your machine, e.g., **MySQL database**.
- You directly set it up and use its client from https://dev.mysql.com/downloads/installer/
- Or you can install a package that gives you access to both MySQL and phpMyAdmin to manage that database using a friendly user interface. For example, https://www.mamp.info/
- Once you have installed MySQL and PhpMyAdmin, you can create your database and the tables you need for your project.

LAB 7 has all the informaiton you need

Database Operations

Create a table with the appropriate schema in MySQL.

- Open phpMyAdmin in MAMP.
- Create a ‘tutorial’ database with a ‘user’ table with the following schema.



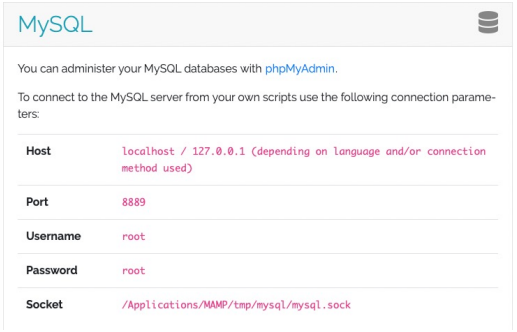
	#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	fname	varchar(100)	utf8_general_ci		No	*None*			Change Drop More
☐	2	email	varchar(200)	utf8_general_ci		No	*None*			Change Drop More
☐	3	aboutus	varchar(20)	utf8_general_ci		No	*None*			Change Drop More

Database Operations

Create the addUser() function in server.js:

```
function addUser(fname, email, aboutus) {
  //connection
  const mysql = require("mysql2");
  let db = mysql.createConnection({
    host: "127.0.0.1",
    user: "root",
    password: "root",
    port: "8889",
    database: "tutorial",
  });

  db.connect(function (err) {
    //SQL command
    let sql =
      "INSERT INTO user (fname, email, aboutus) VALUES ('" +
      fname +
      "', '" +
      email +
      "', '" +
      aboutus +
      "')";

    //exeucte command
    db.query(sql, function (err, result) {
      if (err) throw err;

      //if no errors
      console.log("1 record added");

      //close connection
      db.end();
    });
  });
}
```

Connection info is from MAMP control panel



1. Open connection
2. Create SQL Command.
3. Execute query.
4. Close connection.

Database Operations

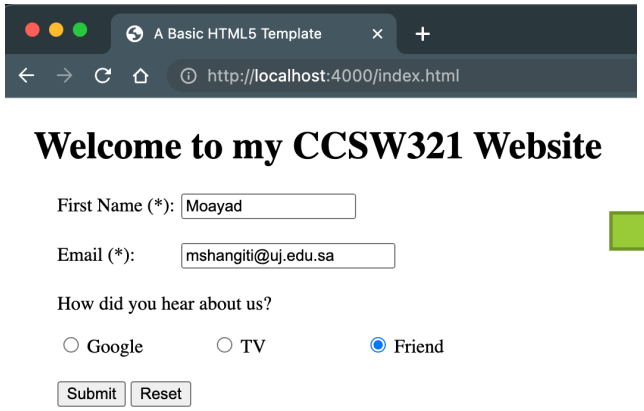
Reply back to client.

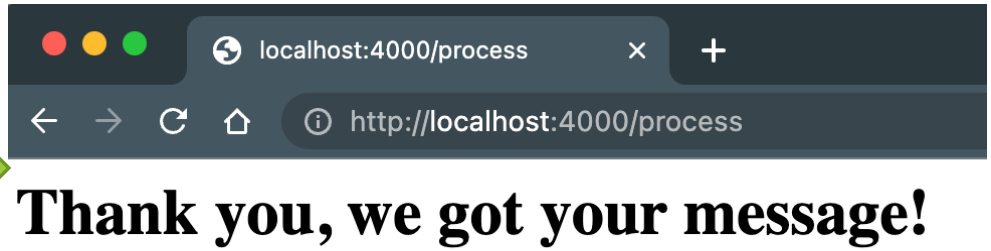
```
const express = require("express");
const app = express();

//static routing
app.use("/", express.static("./website"));

//HTML routing
app.use(express.urlencoded({ extended: false }));
//Handling POST request to /process
app.post("/process", (request, response) => {
  //reading form data
  const fname = request.body.fname;
  const email = request.body.email;
  const aboutus = request.body.aboutus;

  let msg = "";
  //check if all name and are submitted
  if (fname.length > 0 && email.length) {
    //save to database
    addUser(fname, email, aboutus);
    msg = "<h1>Thank you, we got your message! </h1>";
  } else {
    msg =
      "<h1>Kindly complete the form (Name and email are
required!).</h1><h2><a href='index.html'>Click here</a> to
return</h2>";
  }
  //send response
  response.send(msg);
});

//server
app.listen(4000, () => {
  console.log("Server is running.");
});
```

Call the function in your backend code server.js

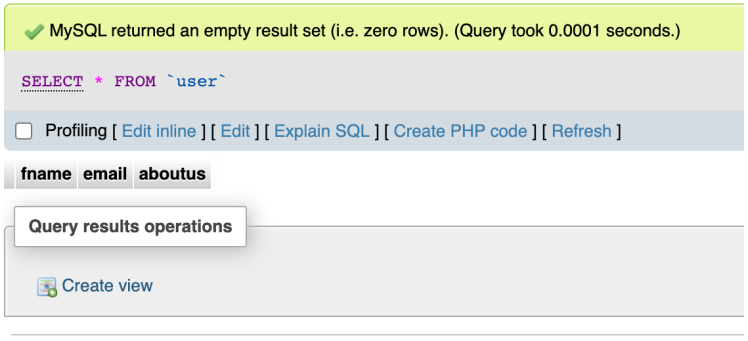
Database Operations

Reply back to client.

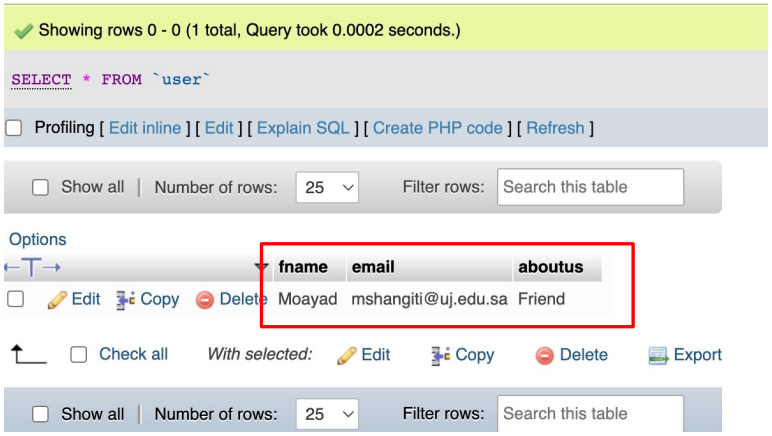




Before INSERT



After INSERT



Backend Data Validation with Node.js

What is Backend data validation?

- Backend data validation is the process of verifying and ensuring that the data submitted to the backend server meets certain criteria or constraints defined by the application.

Backend data validation is important for several reasons:

- **Data integrity**: Validating data helps maintain the integrity of the application's data by ensuring that only valid and meaningful information is stored in the database. It helps prevent data corruption and inconsistencies.

Backend Data Validation with Node.js

Backend data validation is important for several reasons:

- **Security**: Data validation plays a crucial role in ensuring the security of the application. By validating user inputs, it helps prevent common security vulnerabilities such as SQL injection, cross-site scripting (XSS), and other types of attacks that exploit improperly validated data.
- **Business logic enforcement**: Backend data validation allows enforcing business rules and logic specific to the application. It ensures that data entered by users or received from external sources conforms to the expected format, range, or conditions defined by the application's requirements.
- The **most important data validation happens at the backend** because it protects your web application from potential errors or malicious attacks.

Backend Data Validation with Node.js

How to do backend data validation? We need to validate and clean any data sent to the backend before applying any business or data logic.

- Is it a **required** field? If yes, is it included and not empty?
- What is the **expected data type**? E.g., if it is a mobile number, we need to validate that we were sent numbers only.
- What is the **expected length** (should match database column type and length). Does it match minimum/maximum length?
- Is there an **expected format**? E.g., an email has a specific format that should be validated.
- Is there a **whitelist**? E.g., if the user was asked to select from a list, then we need to confirm that the sent data is from the list.

Backend Data Validation with Node.js

How to do backend data validation?

- We can build our own JavaScript backend data validation similar to what we have used for the frontend data validation. However, since the backend data validation is more **critical**, it is **recommended to use external libraries**.
- One of the most used libraries with Node.js is **Express-validator**.
- Before we learn how to use express-validator, you need to know the following commonly used methods:
- **Trim**: Removes the white space before and after a string.
- **Escape**: convert all HTML entitles to their encoded counterpart, e.g., convert "<html>" to "<html&".

Backend Data Validation with Node.js

How does express-validator work?

- **Express-validator** is a middleware module for Express.js. It provides **a set of validation and sanitization functions** that can be used to validate and sanitize user input data before processing it further.
- Install it using “npm install express-validator”.
- Use the check() function to to validate the input data based on criteria such as required fields, data types, lengths, regular expressions, and more.
- It comes with many **built-in** validation and sanitation functions, but you can also build your own **custom checks**. See the documentation for details.
- You can chain multiple validation rules together to create complex validation logic. For example, check('email').isEmail().normalizeEmail().

Backend Data Validation with Node.js

How does express-validator work?

- Along with validation, Express-validator also offers **sanitization** functions that can be used to sanitize user input data, removing any potentially malicious or unwanted content.
- After performing the validation and sanitization, you can access the **validation results** using the **validationResult() function**. It returns an object containing information about any validation errors that occurred. You can then handle these errors as needed, such as displaying error messages to the user or taking appropriate actions based on the validation results.
- It has a great documentation that you can refer to https://github.com/validatorjs/validator.js

Backend Data Validation with Node.js

```
//create the server
const express = require("express");
const app = express();

//static routing
app.use("/", express.static("./website"));

//HTML routing
app.use(express.urlencoded({ extended: false }));
//Handling POST request to /process

//validator
const { check, validationResult } = require("express-validator");
let formValidate = getFormValidation();

app.post("/process", formValidate, (request, response) => {
  const errors = validationResult(request);
  if (!errors.isEmpty()) {
    //errors
    const msg =
      "<h1>Sorry, we found validation errors with your submission
</h1>" +
      printErrors(errors.array()) +
      "<h2><a href='index.html'>Click here</a> to return</h2>";
    response.send(msg);
  } else {
    //no errors
    const fname = request.body.fname;
    const email = request.body.email;
    const aboutus = request.body.aboutus;

    //save to database
    addUser(fname, email, aboutus);

    const msg = "<h1>Thank you, your information has been saved.
</h1>";
    response.send(msg);
  }
});

//server
app.listen(4000, () => {
  console.log("Server is running.");
});
```

printErrors()

Instantiate and connect the express validator.

Create the function **getFormValidation**() with validation rules and add the object as a parameter in the .post() method.

Call the **validationResult**() for errors and reply to the user based on findings.

If errors found, then send back the errors.

If no errors, then proceed.

Will see next the defintion for "**getFormValidation**()" and "**printErrors**()"

Backend Data Validation with Node.js

```
function getFormValidation() {
  //roles
  return [
    check("fname")
      .isLength({ min: 1, max: 100 })
      .withMessage("First name must be between 1 and 100 chars.")
//length
      .isString()
      .withMessage("First name must be a string") //type
      .matches("[A-Za-z]+")
      .withMessage("First name must consist of letters only")
//format
      .trim()
      .escape(),
    //email
    check("email")
      .isEmail()
      .withMessage("Email must of format x@y.z")
      .trim()
      .escape(),
    check("aboutus")
      .custom((val) => {
        const whitelist = ["Google", "TV", "Friend"];
        if (whitelist.includes(val)) return true;
        return false;
      })
      .withMessage(
        "Selection of 'how did you hear about us' must be from the
provided list."
      )
      .trim()
      .escape(),
  ];
}
```

Built-in Checks:

- Check length
- Check type
- Check format

Validate

- Trim

Santize

- Escape

Santize

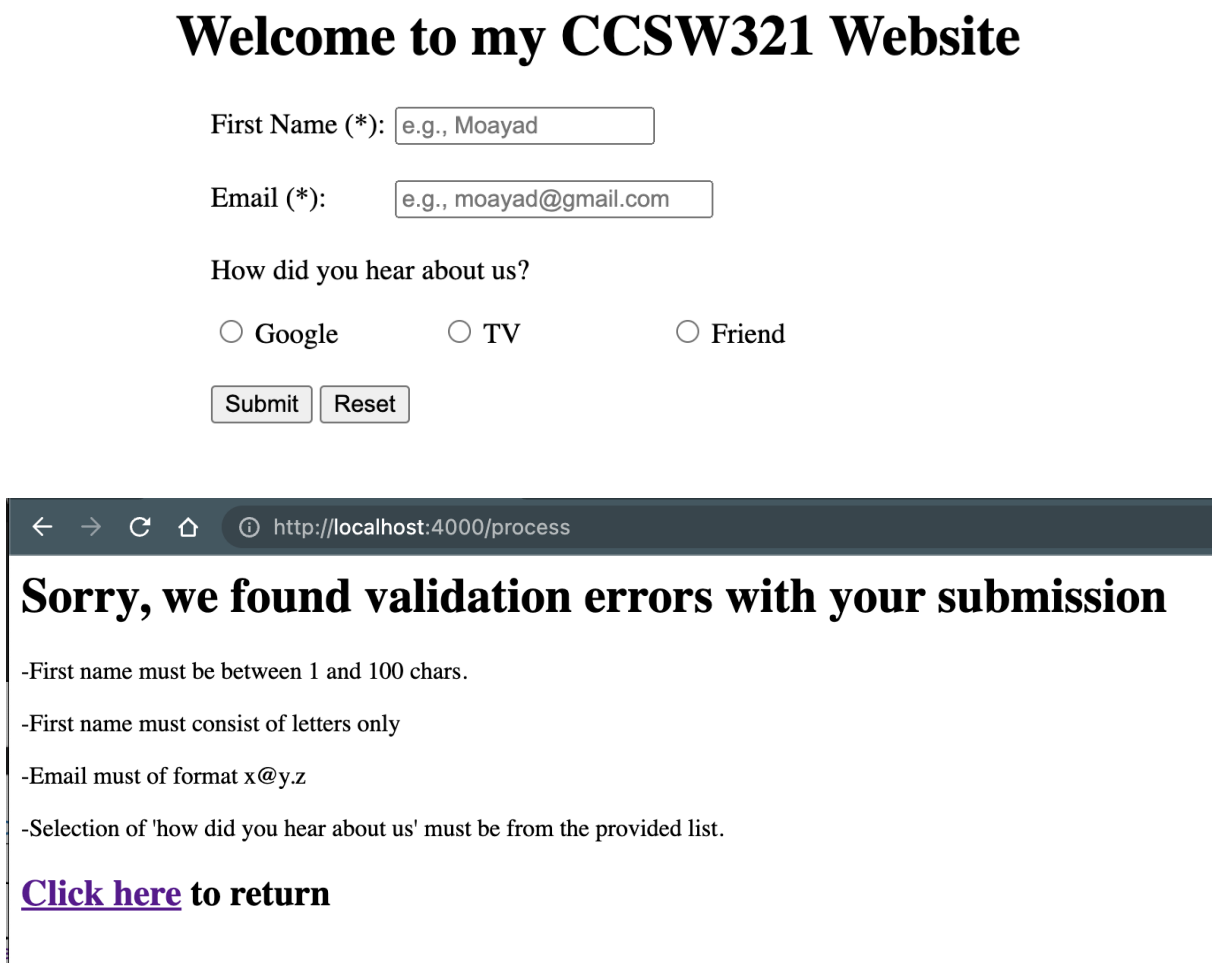
Custom Check:

Check if selection is from whitelist.

Validate

Backend Data Validation with Node.js

What will an empty form submission look like now?



Any questions?
Please feel free to raise your hands and ask.